

Answer key: model commands and expected output. Many tasks have more than one correct answer. Check that the student reached the correct folder and the correct file.

Tidy up the Media Club archive from the terminal. The archive is a mess. Your task is to sort it out with commands only. Do not use the mouse.

You get one zip file. Task 1 gives you the exact commands that unpack it. After that you type every move, copy, rename, search and count. You do not need the mouse at any point in this assignment.

The commands and the messages here are the ones your Ubuntu laptop prints. Ubuntu runs the *bash* shell. Another computer can word the same message a little differently.

The assignment is one tidy-up, done in order. First you look at the mess. Then you build the empty folders that the tidy archive needs. Then you move everything into them. Then you throw away what is left over. Only then do you search, count and join commands together. Do the parts in order: each part works on what the part before it left behind, so a part done out of turn will not have the folders or the files that it needs.

Write your answers on this sheet as you work. The ruled lines, the box in Part 1 and the tables are part of the task.

What is inside `cs17-archive.zip`. Task 1 unpacks the zip. You get one folder with the name `cs17-archive`, and every file and folder in this assignment is inside it. The map of that folder is not printed here. It is inside the archive, in `README.txt`, and Part 1 asks you to print it and copy it out. Keep your copy beside you: most of the work in this assignment is to reach the correct place in that tree.

```
cs17-archive/
|-- README.txt
|-- notes.txt
|-- club-members-attendance-register.txt
|-- inbox/
|   |-- old-phone-photos/
|   |-- Unsorted Camera Dump/
|   |   |-- IMG_0001.jpg
|   |   |-- IMG_0002.jpg
|   |   |-- IMG_0003.jpg
|   |   |-- IMG_0004.jpg
|   |   |-- IMG_0005.jpg
|   |   |-- IMG_0006.jpg
|   |   |-- scoreboard-screenshot.png
|   |   |-- camera-settings-backup.txt
|   |-- competitions/
|   |   |-- photography-competition-entries-2026/
|   |   |   |-- entry-aisha-sunrise.jpg
|   |   |   |-- entry-rohan-bridge.jpg
|   |   |   |-- entry-meera-street-cat.jpg
|   |   |-- entries-received.txt
|-- documents/
|   |-- reports/
|   |   |-- term-1-science-fair-report.txt
|   |   |-- term-2-science-fair-report.txt
|   |-- minutes/
|   |   |-- meeting-minutes-2026-01-14.txt
|   |   |-- meeting-minutes-2026-02-11.txt
|   |-- drafts/
|   |   |-- old-notes.txt
|   |   |-- scrap.txt
|   |-- temp/
|   |   |-- untitled-1.txt
|   |   |-- New Folder/
|-- logs/
|   |-- experiment-log-week-01.txt
|   |-- experiment-log-week-02.txt
|   |-- experiment-log-week-03.txt
|   |-- experiment-log-week-04.txt
```

Three habits. Use them from the first command to the last.

- **Look before you act.** Run `pwd` and `ls` before every command that copies, moves or deletes. There is no Undo. There is no Trash.
- **Press Tab. Do not type the whole name.** Many names in this archive are long. Type three letters, then press Tab.
- **Read the error.** An error message tells you what went wrong. Read the whole line before you type the command again.

Part 1. Look at the mess

Lesson 1

Ask two questions before every command. Where am I? What is here? Each question has one command. You will type these two commands more often than any other command in this assignment. In this part you change nothing. You look, and you write down what you see.

Commands this part uses

<code>pwd</code>	print the absolute path of the folder that you are in
<code>ls</code>	list the contents of this folder
<code>ls -l</code>	list with the size, the date and the permissions
<code>ls -a</code>	list all entries, including the hidden ones
<code>cd folder</code>	go into a folder
<code>cd ..</code>	go up one folder, to the parent
<code>cd ~</code>	go to your home folder
<code>cat file</code>	show the contents of a file on the screen
<code>unzip file.zip</code>	unpack a zip file into the folder that you are in

1. Open the terminal. The archive is in your `Downloads` folder. Run these five commands, in this order. They make a folder on your Desktop, move the zip into it, and unpack it. You meet `mkdir`, `mv` and `cd` again later in this assignment. `unzip` you need only here.

Hint: Add `-q` to `unzip` to unpack the archive quietly, with no list of files.

```
mkdir ~/Desktop/terminal-assignment
mv ~/Downloads/cs17-archive.zip ~/Desktop/terminal-assignment
cd ~/Desktop/terminal-assignment
unzip cs17-archive.zip
ls
```

Expected:

```
unzip prints one `inflating:` line for each file it unpacks. The last command shows
two entries: the zip and the new cs17-archive folder.
If the shell answers `unzip: command not found`, install it once with: sudo apt
install unzip
```

2. Go into the `cs17-archive` folder. Print the folder that you are in. Copy the address exactly as the shell prints it.

Hint: This address is the absolute path of the archive. Task 11 needs it again.

```
cd ~/Desktop/terminal-assignment/cs17-archive
pwd
```

Expected:

```
/home/<your-name>/Desktop/terminal-assignment/cs17-archive
It starts with / and leaves nothing out. A Mac prints /Users/<your-name>/... instead
of /home/<your-name>/..., and the rest is the same.
```

3. List the contents of the archive folder. How many items are there?

```
ls
```

Expected:

```
README.txt  club-members-attendance-register.txt  documents  inbox  logs
notes.txt
6 items. The order depends on the shell. Some shells put the capital R of README.txt
first. Some sort it with the small letters.
```

4. List the folder again with the long details. Name two things that the long listing shows and the plain listing does not.

```
| ls -l
```

Expected:

```
| Any two of these: the size in bytes, the date of the last change, the owner, the
| permissions, and whether the entry is a folder. A folder line starts with d.
```

5. List all entries, including the hidden ones. Write down one entry that the earlier listings did not show.

```
| ls -a
```

Expected:

```
| . and .. , this folder and its parent. A hidden name starts with a dot.
```

6. Show `README.txt` on the screen. Do not open an application. What is the last line?

```
| cat README.txt
```

Expected:

```
| Last tidied: never.
```

Copy the map. `README.txt` holds a map of the archive, and it is the only map you get: your worksheet does not print one. Run `cat README.txt` again and copy the map out by hand, exactly as the terminal prints it. Keep the indentation. A name with a `/` after it is a folder; a name without one is a file. Every part after this one asks you for paths, and your copy is faster to read than `cd`.

```
cs17-archive/
|-- README.txt
|-- notes.txt
|-- club-members-attendance-register.txt
|-- inbox/
|   |-- old-phone-photos/
|   |-- Unsorted Camera Dump/
|       |-- IMG_0001.jpg
|       |-- IMG_0002.jpg
|       |-- IMG_0003.jpg
|       |-- IMG_0004.jpg
|       |-- IMG_0005.jpg
|       |-- IMG_0006.jpg
|       |-- scoreboard-screenshot.png
|       |-- camera-settings-backup.txt
|   |-- competitions/
|       |-- photography-competition-entries-2026/
|           |-- entry-aisha-sunrise.jpg
|           |-- entry-rohan-bridge.jpg
|           |-- entry-meera-street-cat.jpg
|           |-- entries-received.txt
|-- documents/
|   |-- reports/
|       |-- term-1-science-fair-report.txt
|       |-- term-2-science-fair-report.txt
|   |-- minutes/
|       |-- meeting-minutes-2026-01-14.txt
|       |-- meeting-minutes-2026-02-11.txt
|   |-- drafts/
|       |-- old-notes.txt
|       |-- scrap.txt
|   |-- temp/
|       |-- untitled-1.txt
|       |-- New Folder/
|-- logs/
    |-- experiment-log-week-01.txt
    |-- experiment-log-week-02.txt
    |-- experiment-log-week-03.txt
    |-- experiment-log-week-04.txt
```

7. Your map is a photograph of the archive as it was unpacked. From Part 2 on you change the archive, so the map goes out of date. Write down the one command that always tells you the truth about a folder, whatever your map says.

Expected:

```
| ls (with ls -l when you also want the sizes). The map is a note on paper; ls asks the computer.
```

8. Show the Term 1 science fair report. Do not change folder first. Use one command and one relative path. How many rupees came out of the budget?

```
| cat documents/reports/term-1-science-fair-report.txt
```

Expected:

```
| 2400 rupees.
```

9. Start in the archive folder. Go into the reports folder with one command and a relative path. Print the folder to check the result.

```
| cd documents/reports  
| pwd
```

Expected:

```
| /home/<your-name>/Desktop/terminal-assignment/cs17-archive/documents/reports  
| pwd spells the home folder out. It never prints the ~ shortcut, so this is longer than the path you would type.
```

10. Go back up from reports to the archive folder. Use one command.

```
| cd ../../
```

Expected:

```
| Two steps up: reports -> documents -> cs17-archive.
```

11. Start in the archive folder. Go into the deepest folder inside `inbox` with one relative path. Press Tab for the long names.

```
| cd inbox/competitions/photography-competition-entries-2026
```

Expected:

```
| inbox -> competitions -> photography-competition-entries-2026
```

12. Stay in that deep folder. Go to the `logs` folder with a relative path. Do not use `~`. Do not use an absolute path.

```
| cd ../../../logs
```

Expected:

```
| Three steps up to cs17-archive, then down into logs.
```

13. Go to your home folder. Then go back to the archive with one command. Use the absolute path from task 2.

```
| cd ~  
| cd ~/Desktop/terminal-assignment/cs17-archive
```

Expected:

| An absolute path works from any folder, because it does not depend on where you are.

The path drill. Each row asks the same question two times. Write the relative path first. A relative path gives directions from the folder that you are in. Then write the absolute path. An absolute path is the full address and it starts at your home folder `~`. The archive is at `~/Desktop/terminal-assignment/cs17-archive`. Read the rows off the map that you copied.

You are in	You want to reach	Relative path	Absolute path
cs17-archive	logs	logs	~/Desktop/terminal-assignment/cs17-archive/logs
cs17-archive/logs	cs17-archive/documents	../documents	~/Desktop/terminal-assignment/cs17-archive/documents
documents/reports	inbox	../../inbox	~/Desktop/terminal-assignment/cs17-archive/inbox
inbox/Unsorted Camera Dump	logs	../../logs	~/Desktop/terminal-assignment/cs17-archive/logs
inbox/competitions/ photography- competition-entries- 2026	documents/minutes	../../../documents/ minutes	~/Desktop/terminal-assignment/cs17-archive/documents/ minutes
documents/minutes	documents/reports	../reports	~/Desktop/terminal-assignment/cs17-archive/documents/ reports
documents/drafts	cs17-archive	..	~/Desktop/terminal-assignment/cs17-archive
anywhere at all	your home folder	(no relative path. It depends on where you are)	~

14. Read your finished table. Write one sentence about when to use a relative path. Write one sentence about when to use an absolute path.

Expected:

A relative path is shorter and it still works after somebody moves or renames the whole archive. Use it for short steps inside one project.
An absolute path always means the same one folder, from anywhere. Use it to jump in from another part of the computer.

Remember

`..` is not a normal folder. It is a shortcut and it always means the folder above this one. `.` means this folder. `ls -a` shows both of them.

Part 2. Build the shelves

Lesson 1

You cannot tidy a room with no shelves in it. The archive gives you nowhere to put anything: there is no gallery, no backup and no sorted folder. You make them, before you move a single file. One command does it, and it is the same command in every part of this assignment. Start in the archive folder and check with `pwd`.

Commands this part uses

<code>mkdir name</code>	make a new folder
<code>mkdir a b c</code>	make several folders with one command
<code>mkdir one/two</code>	make two inside one, when one already exists
<code>mkdir -p one/two/three</code>	make every folder in the path that is missing
<code>ls</code>	check what you made
<code>ls -l</code>	a folder line starts with d

1. Make one folder in the archive folder with the name `gallery`. Then list the archive folder to check it.

```
mkdir gallery
ls
```

Expected:

```
gallery is now in the listing. The archive holds 7 items.
```

2. Now try to make a folder inside a folder that does not exist yet. Run `mkdir sorted/notes`. Copy the message. Why did the command fail?

```
mkdir sorted/notes
```

Expected:

```
mkdir: cannot create directory 'sorted/notes': No such file or directory
There is no sorted folder. mkdir makes one folder, the last one in the path, and it
will not invent the folders above it.
```

3. Run the same command again with the `-p` option. Then list the archive folder and `sorted`. How many folders did `-p` make, and which ones?

```
mkdir -p sorted/notes
ls
ls sorted
```

Expected:

```
Two: sorted, and notes inside it. -p makes every folder in the path that is missing,
in one go.
Part 3 moves the club's notes into sorted/notes.
```

4. Two folders are still missing: `backup`, for a copy of the experiment logs, and `competition-2026` inside `gallery`, for the competition photographs. Make both with *one* command. You do not need `-p`: every folder above them already exists.

```
mkdir backup gallery/competition-2026
ls
ls gallery
```

Expected:

```
mkdir takes as many names as you give it and makes every one of them, so one command
is enough.
gallery now holds competition-2026 and nothing else.
```

5. Run `mkdir gallery` a second time. Copy the message. Then run `mkdir -p gallery` and say what is different.

```
mkdir gallery
mkdir -p gallery
```

Expected:

```
mkdir: cannot create directory 'gallery': File exists
mkdir refuses, which is the safe answer: it will not touch a folder that is already
there and it will not empty it.
mkdir -p prints nothing and changes nothing. -p treats an existing folder as a job
already done.
```

6. List the archive folder. Write down the names of the three folders that were not in the map you copied in Part 1. Then say why `sorted/notes` is not one of the names you wrote.

```
ls
ls sorted
ls gallery
```

Expected:

```
gallery, backup and sorted are the three new names. The rest of the listing is what
came in the zip, unchanged.
sorted/notes is not in it because ls lists one folder, not the folders inside it.
```

7. Look at your map from Part 1 again. It has two folders in it that nothing in this assignment ever uses: `inbox/old-phone-photos` and `documents/temp`. List both of them and write down what is inside each one.

```
ls inbox/old-phone-photos
ls documents/temp
```

Expected:

```
inbox/old-phone-photos is empty: ls prints nothing at all.
documents/temp holds untitled-1.txt and a folder called New Folder, which is itself
empty. Somebody made them years ago and never came back.
These are the leftovers. Part 4 deletes them. Leave them alone until then: you cannot
tell rubbish from a misplaced file until everything worth keeping has been moved.
```

Remember

Make the folders before you move anything into them. `mv` and `cp` do not create a destination: if you move a file into a folder name that does not exist, the shell quietly renames the file to that name instead, and the file is not where you think it is.

Part 3. Move everything into place

Lesson 2 and 3

The shelves are up. Now carry the archive onto them. Two commands do all of it. `mv` moves a file and leaves nothing behind, which is what tidying up means. `cp` makes a second copy and keeps the first, which is what a backup means. Three shortcuts make the typing bearable: Tab finishes a name, quotes hold a name with spaces together, and `*` stands for a whole group of files. Start each task in the archive folder.

Commands this part uses

Tab	finish the name that I started to type
"name with spaces"	quote a name so the shell reads it as one name
*	match any text here. *.jpg means every name that ends in .jpg
mv from to	move a file, or rename it
cp from to	copy a file and keep the original
mv a b c folder/	move several files into a folder with one command
ls folder	check the destination afterwards

1. Start in the archive folder. Type `cat club` and stop there. Press Tab. Write the full name that the shell completes. Then press Enter and look at the file.

```
| cat club<Tab>      ->    cat club-members-attendance-register.txt
```

Expected:

```
| club-members-attendance-register.txt. You typed 8 characters and not 38. You also  
| cannot spell the name wrong.
```

2. Go into `documents/minutes`. Type `cat meeting-minutes-2026-0` and press Tab. Nothing happens. Press Tab a second time. Explain why the first Tab did not complete the name.

```
| cd documents/minutes  
| cat meeting-minutes-2026-0<Tab><Tab>
```

Expected:

```
| Two files start with those letters, ...-2026-01-14.txt and ...-2026-02-11.txt. The  
| shell cannot know which one you mean. The second Tab lists both files. You then type  
| one more character and press Tab again.
```

3. Go back to `inbox`. Enter the `Unsorted Camera Dump` folder in three ways. Write the result of each one. (a) Type the name with no quotes and no Tab, then copy the error. (b) Put the whole name in double quotes. (c) Type `Uns` and press Tab, then copy what Tab typed for you.

```
| cd ~/Desktop/terminal-assignment/cs17-archive/inbox  
| cd Unsorted Camera Dump      # fails  
| cd "Unsorted Camera Dump"    # works  
| cd Uns<Tab>                  # becomes: cd Unsorted\ Camera\ Dump
```

Expected:

```
| (a) bash: cd: too many arguments  
| The shell split the name at the spaces. It read three separate arguments.  
| (b) This works. The quotes join the words into one name.
```

(c) Tab puts a backslash before each space. A backslash does the same job as the quotes.

4. Go back to the archive folder. List every `.jpg` file inside the camera dump without going into it. How many are there? Which two files in that folder does the pattern miss?

```
cd ~/Desktop/terminal-assignment/cs17-archive
ls inbox/"Unsorted Camera Dump"/*.jpg
```

Expected:

6 files, `IMG_0001.jpg` to `IMG_0006.jpg`.
The pattern misses `scoreboard-screenshot.png` and `camera-settings-backup.txt`. A `.png` and a `.txt` do not end in `.jpg`.

5. Move every one of those `.jpg` files into `gallery` with one command. Then list both folders. Why is the camera dump not empty yet?

```
mv inbox/"Unsorted Camera Dump"/*.jpg gallery/
ls gallery
ls inbox/"Unsorted Camera Dump"
```

Expected:

`gallery` now holds 6 photographs and the `competition-2026` folder.
The dump still holds the `.png` and the `.txt`, because `*.jpg` did not match them. `mv` moved the files and left nothing behind, so the photographs are in one place only.

6. The screenshot is a picture too. Move `scoreboard-screenshot.png` into `gallery`. Press Tab instead of typing that name.

```
mv inbox/"Unsorted Camera Dump"/scoreboard-screenshot.png gallery/
```

Expected:

`gallery` now holds 7 pictures and the `competition-2026` folder.

7. `camera-settings-backup.txt` is not a picture. It is a note. Move it into `sorted/notes`. Then list the camera dump. What does the shell print for an empty folder?

```
mv inbox/"Unsorted Camera Dump"/camera-settings-backup.txt sorted/notes/
ls inbox/"Unsorted Camera Dump"
```

Expected:

Nothing at all. `ls` prints an empty line for an empty folder: no message, no error. The folder itself is still there, and Part 4 deletes it.

8. Move the three competition entries into `gallery/competition-2026`. Use one command and a `*` pattern. Leave `entries-received.txt` where it is: it is the record of who entered, not a photograph.

```
mv inbox/competitions/photography-competition-entries-2026/*.jpg gallery/competition-2026/
ls gallery/competition-2026
ls inbox/competitions/photography-competition-entries-2026
```

Expected:

3 entries in `gallery/competition-2026`.
`entries-received.txt` is the only thing left in the competition folder.

9. The experiment logs must stay in `logs` and a copy must go into `backup`. Use one command. Which of `mv` and `cp` is the correct one here, and why?

```
cp logs/*.txt backup/
ls logs
ls backup
```

Expected:

```
cp. All 4 logs are now in both folders. mv would have emptied logs, and a backup that
is the only copy is not a backup.
```

10. Move `old-notes.txt` from `documents/drafts` into `sorted/notes`.

```
mv documents/drafts/old-notes.txt sorted/notes/
```

Expected:

```
The file is no longer in drafts. It is now in sorted/notes.
```

11. Rename that file to `last-year-notes.txt`. Keep it in `sorted/notes`.

```
mv sorted/notes/old-notes.txt sorted/notes/last-year-notes.txt
```

Expected:

```
A rename uses the same command as a move. mv changes the name, the place, or both.
```

12. Copy the Term 2 report into `sorted/notes`. The original must stay in `documents/reports`.

```
cp documents/reports/term-2-science-fair-report.txt sorted/notes/
ls documents/reports
ls sorted/notes
```

Expected:

```
The report is now in both folders. cp does not change the original.
```

13. You used `mv` for the photographs and `cp` for the logs and the report. Write the difference in one sentence. Then write which command a tidy-up needs and which command a backup needs.

Expected:

```
cp keeps the original and makes a second copy. mv leaves nothing behind.
A tidy-up needs mv: a tidy archive does not keep the same photograph in two places. A
backup needs cp.
```

14. List `sorted/notes` with the long details. Which file is the biggest? How do you know?

```
ls -l sorted/notes
```

Expected:

```
term-2-science-fair-report.txt. Its number in the size column is the largest.
```

Remember

The shell expands `*`, not the command. The shell hands `mv` all six filenames before `mv` starts. For this reason, run `ls` with a pattern before you use that same pattern with a command that moves or deletes: `ls` shows you exactly what the other command is about to receive.

Everything worth keeping is on a shelf. What is left is either rubbish or an empty folder that nothing lives in. Deleting is the last step of a tidy-up and never the first, because before the move you cannot tell a rubbish file from a file in the wrong place. There is no Trash and there is no Undo, so every delete in this part comes after `pwd` and `ls`.

Commands this part uses

<code>rm file</code>	delete a file. There is no Trash and no Undo
<code>rmdir folder</code>	delete a folder, but only if the folder is empty
<code>pwd</code>	run this before a command that deletes. Where am I?
<code>ls</code>	run this before a command that deletes. Is that the right file?
<code>command --help</code>	ask a command to list its own options
<code>man command</code>	open the full manual. Press q to quit

1. `documents/drafts` still holds `scrap.txt`, and the file says `delete me`. Run the two check commands from the habits box first, then delete it. Write all three commands on the lines below.

```
pwd
ls documents/drafts
rm documents/drafts/scrap.txt
```

Expected:

```
pwd shows that you are in cs17-archive. ls shows that the file is the correct one.
Then rm deletes it, permanently.
```

2. `documents/drafts` is now empty. Delete the empty folder.

```
ls documents/drafts
rmdir documents/drafts
```

Expected:

```
drafts disappears. rmdir works because the folder is empty: you moved old-notes.txt
out in Part 3 and deleted scrap.txt a moment ago.
```

3. Part 3 emptied the camera dump. Delete that folder. The name has spaces in it, so quote it or press Tab.

```
rmdir inbox/"Unsorted Camera Dump"
ls inbox
```

Expected:

```
competitions, and nothing else. The mess that the whole assignment started with is
gone, and no file was lost: every one of them is on a shelf you made in Part 2.
```

4. `inbox/old-phone-photos` came with the archive and has been empty since the day somebody made it. Delete it. This is the only folder on this sheet that needs no preparation at all.

```
ls inbox/old-phone-photos
rmdir inbox/old-phone-photos
ls inbox
```

Expected:

```
ls prints nothing, so the folder is empty and rmdir accepts it straight away.
inbox is left with Unsorted Camera Dump and competitions.
```

5. `documents/temp` is the other leftover. It holds `untitled-1.txt`, which says `asdf`, and an empty folder called `New Folder`. Nobody has touched either for years. Empty the folder: delete the file, then delete `New Folder`. Its name has a space in it.

```
ls documents/temp
rm documents/temp/untitled-1.txt
rmdir documents/temp/"New Folder"
```

Expected:

```
The quotes hold the two words together, exactly as they did for Unsorted Camera Dump
in Part 3. Tab does the same job with a backslash.
documents/temp is now empty.
```

6. Now delete `documents/temp` itself. You could not have run this command two tasks ago. Write down why, in one sentence.

```
rmdir documents/temp
ls documents
```

Expected:

```
reports, minutes, drafts.
rmdir refuses a folder that is not empty, so the file and the folder inside temp had
to go first. A folder tree comes apart from the inside out. (rmdir -p
documents/temp/'New Folder' would have done the last two steps in one command, for the
same reason that mkdir -p made two folders in Part 2.)
```

7. Try to delete `documents` with the same command. Copy the message from the shell. Explain why the command failed, and why this refusal is a good thing.

```
rmdir documents
```

Expected:

```
rmdir: failed to remove 'documents': Directory not empty
rmdir deletes empty folders only. This is a safety rule. It stops one short command
from taking the reports and the minutes with the folder.
```

8. Try to delete the competition folder, `inbox/competitions/photography-competition-entries-2026`. It fails. Read the message, then answer: should you make it empty so that the command works?

```
rmdir inbox/competitions/photography-competition-entries-2026
```

Expected:

```
rmdir: failed to remove '...photography-competition-entries-2026': Directory not empty
No. entries-received.txt is still in there and it is the record of who entered the
competition. The folder is not left over, it is in use. rmdir refusing is the computer
telling you that you have not finished thinking.
```

9. The scoreboard screenshot is not a photograph of the club. It is a picture of a screen and nobody wants it in the gallery. Delete `gallery/scoreboard-screenshot.png`. Write the two check commands that come before the delete, then run all three.

```
pwd
ls gallery
rm gallery/scoreboard-screenshot.png
```

Expected:

gallery now holds 6 photographs and the competition-2026 folder.
 This one is permanent in a way the others were not: you moved this file in Part 3, so there is no second copy anywhere. pwd and ls first, every time.

10. Run `ls --help`. The page is long. Do not read all of it. Find the line for one option that you already use, and write that line down.

```
| ls --help
```

Expected:

```
| The first line is `Usage: ls [OPTION]... [FILE]...`. Then comes one line for each
| option, for example:
|   -a, --all          do not ignore entries starting with .
|   -l                 use a long listing format
| Not every command has --help. On a Mac, ls does not. Use man when --help fails.
```

11. Open the manual for `rmdir`. Find out what `-p` does there. Write the answer in your own words. Press `q` to quit.

```
| man rmdir
```

Expected:

```
| -p removes the folder and then its parent folders too, as long as each one becomes
| empty. It is the delete that matches mkdir -p.
```

Break it on purpose. Start in the archive folder. Run each line exactly as it is written. Copy the important part of the message from the shell. Then write in a few words why the command failed. Every line here fails safely: none of them deletes anything.

Run this	What the shell printed	Why it failed
<code>cd documets</code>	<code>bash: cd: documets: No such file or directory</code>	The folder name is mistyped. Tab prevents this.
<code>cd notes.txt</code>	<code>bash: cd: notes.txt: Not a directory</code>	notes.txt is a file. You cannot go inside a file.
<code>cat notes</code>	<code>cat: notes: No such file or directory</code>	'txt' is part of the name. The shell does not add it.
<code>cp missing.txt backup/</code>	<code>cp: cannot stat 'missing.txt': No such file or directory</code>	You cannot copy a file that is not there.
<code>rm gallery</code>	<code>rm: cannot remove 'gallery': Is a directory</code>	rm deletes files. A folder needs rmdir.
<code>rmdir logs</code>	<code>rmdir: failed to remove 'logs': Directory not empty</code>	rmdir deletes empty folders only.
<code>mkdir sorted</code>	<code>mkdir: cannot create directory 'sorted': File exists</code>	The folder is already there. mkdir will not touch it.

12. Capital letters, part one. This part is the same on every computer. Run `ls -l`. Then run `ls -L`. The letter is the same and the case is different. Are the two listings the same?

```
ls -l
ls -L
```

Expected:

```
No. -l gives the long listing. -L is a different option and it follows shortcuts, so
you get the short listing back.
The command reads its own options, not the filesystem. So options are case sensitive
on every computer. The word that you give to grep is case sensitive too.
```

13. Capital letters, part two. Names. Run `cat NOTES.txt`. Then run a command name in capitals, `LS`. Copy both messages. The file `notes.txt` is there and the command `ls` exists, so why does Ubuntu refuse both?

```
cat NOTES.txt
LS
```

Expected:

```
cat: NOTES.txt: No such file or directory
bash: LS: command not found
Ubuntu treats NOTES.txt and notes.txt as two different names, and LS and ls as two
different commands.
A Mac or a Windows laptop often accepts both, because it ignores capitals in names.
Ubuntu never does, and neither does a server. Type every name the way ls shows it.
```

14. A command failed and you do not know why. Write the five checks that you make, in order, before you ask for help.

Expected:

1. Where am I? -> `pwd`
2. What is here? -> `ls`
3. Is the name exact? -> check the spelling, the capitals and the spaces
4. What does the command say? -> `--help` or `man`
5. Run the command again with the corrected path or name.

Remember

You can find `rm -rf` on the internet. Do not use it. It deletes a whole folder tree and it asks no questions. In the wrong folder it deletes your work. A tidy-up done in the right order never needs it: move everything worth keeping first, and what is left is empty folders and rubbish that `rm` and `rmdir` handle one at a time.

Part 5. Find things and count them

Lesson 3

The archive is tidy. Now use it. Two commands answer almost every question about a text file without opening it: `grep` shows the lines that contain a word, and `wc` counts. The shell also remembers every command that you have typed today, which saves you typing the long ones a second time. Start in the archive folder.

Commands this part uses

<code>history</code>	list the commands that you ran
<code>Up / Down</code>	move back and forward through earlier commands
<code>clear</code>	clear the screen. <code>Ctrl+L</code> does the same
<code>head file</code>	show the first 10 lines of a file
<code>tail file</code>	show the last 10 lines of a file
<code>head -3 file</code>	show the first 3 lines. <code>tail -3</code> shows the last 3
<code>grep word file</code>	show every line of the file that contains word
<code>grep word *.txt</code>	search a whole group of files at once
<code>wc file</code>	count the lines, words and bytes
<code>wc -l file</code>	count the lines only

1. Show the commands that you have run so far. How many commands did the shell record? Then press the Up arrow to find the `pwd` from Part 1 and run it again without typing it.

```
| history
```

Expected:

```
| The number at the left of the last line. Up and Down move through the same list. You  
| can also edit a line before you press Enter.  
| Keep this terminal window open. Part 7 saves this list into a file and hands it in,  
| and a new window starts a new list.
```

2. Clear the screen. Your commands are still there. Press the Up arrow to check.

```
| clear
```

Expected:

```
| clear clears the display only. The history does not change.
```

Predict before you press Enter. Start in the archive folder. Write what you expect each line to print. Then run the line and correct your answer. The archive is the tidy one that Parts 2 to 4 left behind.

Command	Write your prediction, then run the line
<code>ls *.txt</code>	the 3 .txt files in the archive folder itself. The shell does not search the subfolders
<code>ls *.jpg</code>	ls: cannot access '*.jpg': No such file or directory. Nothing in the archive folder matches, so bash hands the * to ls unchanged
<code>ls gallery/*.jpg</code>	the 6 photographs that you moved in Part 3, with gallery/ in front of each name
<code>ls inbox/*</code>	photography-competition-entries-2026. inbox holds only competitions now, and ls lists what is inside it

Command	Write your prediction, then run the line
<code>ls logs/*week-0*</code>	all 4 experiment logs. A * can also stand in the middle of a pattern

3. The attendance register is far too long to read on the screen. Show the beginning of it with `head`. Do not give the command any number. How many lines did it print, and what is the first one?

```
| head club-members-attendance-register.txt
```

Expected:

```
| 10 lines. That is what head prints when you do not ask for a number.
| The first line is the header, date,member,status, which names the three columns. The
| tenth is 2026-01-14,Meera,present.
```

4. Show the first three lines only. Then show the last three lines. Write down the date on the last line of the file.

```
| head -3 club-members-attendance-register.txt
| tail -3 club-members-attendance-register.txt
```

Expected:

```
| head -3:
| date,member,status
| 2026-01-07,Aisha,absent
| 2026-01-07,Rohan,present
|
| tail -3:
| 2026-02-11,Zaid,present
| 2026-02-11,Kavya,absent
| 2026-02-11,Ibrahim,present
|
| The last meeting in the register is 2026-02-11. head counts from the top, tail counts
| from the bottom, and the number after the dash is how many lines you want.
```

5. In Part 1 you used `cat` to find the last line of `README.txt` and then read down the whole file to get to it. Print that line again with one command that shows nothing else.

```
| tail -1 README.txt
```

Expected:

```
| Last tidied: never.
| tail -1 is the shortest way to ask a file how it ends. cat had to print the whole map
| first.
```

6. Look at the top of each experiment log with one command. `head` takes a * pattern like the other commands. How does it keep the four answers apart?

```
| head -1 logs/*.txt
```

Expected:

```
| It prints a ==> filename <== heading above each file, the same idea as the filename
| grep puts in front of a matching line.
| The four headings are the week 1 to week 4 title lines.
```

7. Write one sentence. The logs are short, so `cat` works on them. Why would you still reach for `head` or `tail` on a file you do not know?

Expected:

```
| Because cat prints the whole file however long it is. On a file with thousands of  
| lines that floods the screen and scrolls the part you wanted out of sight. head and  
| tail show you the shape of a file first, and cost nothing on a short one.
```

8. Search the week 1 experiment log for the word `SUCCESS`. How many days passed?

```
| grep SUCCESS logs/experiment-log-week-01.txt
```

Expected:

```
| 3 lines. grep shows the whole line, not the word only.
```

9. Search all four logs with one command. Which week has the most days with `SUCCESS`?

```
| grep SUCCESS logs/*.txt
```

Expected:

```
| Week 3, with 4 days. With more than one file, grep puts the filename in front of every  
| matching line.
```

10. Search the Term 2 report for `Scratch` with a capital S. Then search it for `scratch` with a small s. Are the lines the same? What does this show about `grep`?

```
| grep Scratch documents/reports/term-2-science-fair-report.txt  
| grep scratch documents/reports/term-2-science-fair-report.txt
```

Expected:

```
| Capital S gives 2 lines, about the programming. Small s gives 1 line, about the camera  
| lens.  
| grep is case sensitive on every computer, whatever the filesystem does with filenames.  
| The two searches are two different questions.
```

11. Count the lines, words and bytes in the attendance register. Write the three numbers. Which number is the count of register entries?

```
| wc club-members-attendance-register.txt
```

Expected:

```
| The first number is the line count, 31. The register holds 30 entries and 1 header  
| line.
```

12. Count the lines only. Then find how many lines contain `Aisha`. Count them by hand from the output of `grep`.

```
| wc -l club-members-attendance-register.txt  
| grep Aisha club-members-attendance-register.txt
```

Expected:

```
| 31 lines in total. 5 of them contain Aisha, one line for each meeting date.
```

13. Count the lines in all four logs with one command. What is the total? Where does the shell show the total?

```
| wc -l logs/*.txt
```

Expected:

```
6 lines in each log and 24 in total. wc adds a `total` line at the bottom when you
give it several files.
```

14. Your backup was made with `cp` in Part 3. Count the lines in `backup` the same way. Should the total match the one from the task before?

```
wc -l backup/*.txt
```

Expected:

```
Yes, 24 again. cp copies the contents, so a backup that does not match the original is
a backup that went wrong.
```

Remember

`grep` searches inside files. `ls` with a `*` searches names. Two different questions: *which file is called this* and *which file says this*.

Part 6. Join commands up: pipes, files and a cow

Lesson 4

Each command that you know does one small job. `cat` shows a file. `grep` selects lines. `wc` counts. The shell joins these jobs end to end, so that the output of one command becomes the input of the next command:

```
INPUT → COMMAND → OUTPUT
```

`|` passes the output to another command. `>` writes the output into a file instead of the screen. `>>` adds the output to the end of a file. This is also where the files that you hand in come from: every one of them is the output of a command, caught in a file instead of printed. Predict the result of each line before you press Enter.

Commands this part uses

<code>a b</code>	give the output of a to b as input
<code>a > file</code>	write the output of a into file and replace the contents
<code>a >> file</code>	add the output of a to the end of file and keep the rest
<code>touch file</code>	make an empty file
<code>echo "text"</code>	show some text on the screen
<code>cowsay text</code>	draw a cow saying text

1. Show the week 1 experiment log. Pass the output to `grep` so that only the `SUCCESS` lines appear.

```
cat logs/experiment-log-week-01.txt | grep SUCCESS
```

Expected:

```
The same 3 lines as in Part 5. This time grep did not open the file. It read the
output of cat.
```

2. Add one more stage. The shell must print the number of successful days and not the lines.

```
cat logs/experiment-log-week-01.txt | grep SUCCESS | wc -l
```

Expected:

```
3. Three commands do three small jobs: show the file, keep the matching lines, count
what is left.
```

3. Write one sentence. What does the `|` do in that command?

Expected:

```
It takes the output of the command on its left and gives it to the command on its
right as input, instead of showing it on the screen.
```

4. Make an empty file in the archive folder with the name `tidy-up-log.txt`. Then show that it is there and that it is empty.

```
touch tidy-up-log.txt
ls -l tidy-up-log.txt
```

Expected:

```
The size column says 0. touch makes a file with no contents at all.
```

5. Write one line into that file. The line has your name and the word *tidied*. Then show the file to check it.

```
echo "Archive tidied by Aisha" > tidy-up-log.txt
cat tidy-up-log.txt
```

Expected:

```
Archive tidied by Aisha
The > sends the output of echo into the file and not to the screen. It replaces the
old contents of the file.
```

6. Save the listing of `gallery` into a file with the name `gallery-list.txt`. Put the file in the archive folder. Then show the file. How many lines does it have, and why is one of them not a photograph?

```
ls gallery > gallery-list.txt
cat gallery-list.txt
```

Expected:

```
7 lines: the 6 photographs and the competition-2026 folder. ls lists a folder by name
like anything else.
The first command printed nothing on the screen, because the output went into the
file.
```

7. Add one line to the end of that file with the name of the person who checked it. Do not delete the lines that are already there. Then show the file.

```
echo "checked by Aisha" >> gallery-list.txt
cat gallery-list.txt
```

Expected:

```
8 lines now: the listing, then your line.
```

8. Run the same line again with a single `>` and not `>>`. Then show the file. What happened to the names? Build the file again in the correct way. Part 7 hands this file in.

```
echo "checked by Aisha" > gallery-list.txt
cat gallery-list.txt
# build it again:
ls gallery > gallery-list.txt
echo "checked by Aisha" >> gallery-list.txt
```

Expected:

```
> deleted the whole file and wrote one line. >> adds and > replaces. This is the most
common way to lose a file by accident.
```

9. How many test days failed in all four logs? Answer with one command. Use `cat`, `grep` and `wc`.

```
cat logs/*.txt | grep FAILED | wc -l
```

Expected:

```
7.
```

10. How many lines of the attendance register contain `Aisha`? Do not print the lines. Save only the number into a file with the name `aisha-count.txt`. Then show that file.

```
grep Aisha club-members-attendance-register.txt | wc -l > aisha-count.txt
cat aisha-count.txt
```

Expected:

```
5. grep finds the lines, wc -l counts them, and > puts the count in the file and not
on the screen.
```

11. Now something that is not installed yet. `cowsay` draws a cow with a speech bubble. Install it once with the first line, then run the second. The install asks for your password and prints nothing while you type it.

```
sudo apt install cowsay
cowsay "CS17 Army ROCKS"
```

Expected:

```
-----
< CS17 Army ROCKS >
-----
      \   ^__^
       \  (oo)\_______
          (__)\       )\/\
              ||----w |
              ||     ||
```

The bubble grows to fit whatever you give it, and the cow is drawn underneath in plain characters.

If apt answers ``Unable to locate package``, run ``sudo apt update`` once and install again.

`cowsay` installs into `/usr/games`, which is on the PATH of an Ubuntu desktop. If the shell still answers ``cowsay: command not found`` straight after a successful install, run it as `/usr/games/cowsay`.

12. `cowsay` reads its input the same way `grep` and `wc` do, so it can be the last stage of a pipeline. Make the cow say how many things are in `gallery`. Use `ls`, `wc -l` and `cowsay`, joined with two `|`.

```
ls gallery | wc -l | cowsay
```

Expected:

```
A cow saying 7.
```

```
Nothing in the first two commands knows that a cow exists. Each one writes its output,
and the shell hands that output to whatever comes next.
```

13. A cow is text like any other text, so it can go into a file. Send the cow to the end of `tidy-up-log.txt` instead of the screen, then show the file. Your name line from task 5 must still be at the top.

```
cowsay "The archive is tidy" >> tidy-up-log.txt
cat tidy-up-log.txt
```

Expected:

```
Your line first, then the cow, drawn in the file exactly as it was drawn on the
screen. >> added it and kept what was already there.
```

One more prediction round. Write your answer first. Then run the line and compare it with your answer.

Command	Write your prediction, then run the line
<code>cat logs/*.txt wc -l</code>	24
<code>grep present club-members-attendance-register.txt wc -l</code>	22
<code>grep absent club-members-attendance-register.txt wc -l</code>	8
<code>cat documents/reports/*.txt grep budget wc -l</code>	2
<code>ls gallery wc -l</code>	7

Remember

Read a pipeline from left to right, like a sentence: *show the logs, keep the failures, count them*. If a long pipeline gives a strange result, remove the last stage and run it again. This shows you what passes through the middle.

Part 7. Save your history and hand in

Lesson 4

Everything that you typed in this assignment is still in the shell's history, and `history` prints it. A printed list is no use to anybody, so you send it into a file instead, with the `>` from Part 6. That file is what you hand in with the archive: it is the record of how you tidied it, in your own commands. Do this part last, in the same terminal window that you have used all along.

Commands this part uses

<code>history</code>	print every command that you ran in this window
<code>history > file</code>	put that list into a file instead of on the screen
<code>head -20 file</code>	show the first 20 lines of a file
<code>tail -20 file</code>	show the last 20 lines of a file
<code>mv a b c folder/</code>	move several files into a folder with one command
<code>zip -r name.zip folder</code>	pack a folder and everything inside it

1. Start in the archive folder and check it with `pwd`. Print your command history one more time. It is long now, so show only the last twenty lines. Use `history`, a `|` and `tail`.

```
| pwd
| history | tail -20
```

Expected:

```
| The last twenty commands, each with its number. history writes its list out like any
| other command, so tail can cut it down.
```

2. Save the whole history into a file with the name `history.txt`, in the archive folder. Nothing must appear on the screen. Then count the lines in the file.

```
| history > history.txt
| wc -l history.txt
```

Expected:

```
| One line for each command that you have run in this window since you opened it. The
| number is your own: a student who mistyped more commands has more lines, and that is
| fine.
| If the file is nearly empty, you are in a terminal window that you opened later. The
| history belongs to the window, not to the computer.
```

3. Show the first ten lines of `history.txt`. Which command from Part 1 is at the top?

```
| head -10 history.txt
```

Expected:

```
| The mkdir, mv, cd and unzip lines from task 1 of Part 1, in the order that you ran
| them.
```

4. How many folders did you make in this whole assignment? Answer it from the file, with one command: search `history.txt` for `mkdir` and count the matching lines.

```
| grep mkdir history.txt | wc -l
```

Expected:

```
| Your own number. It counts the failed attempts too, because the shell records what you
| typed and not what worked. Part 2 alone had two mkdir commands that failed on purpose.
```

5. Make a folder with the name `submission` inside `cs17-archive`.

```
| mkdir submission
```

Expected:

```
| The last folder you make in this assignment, and the same command as the first one in
| Part 2.
```

6. Move `gallery-list.txt`, `aisha-count.txt` and `tidy-up-log.txt` into it. Use one command for all three files. Leave `history.txt` in the archive folder.

```
| mv gallery-list.txt aisha-count.txt tidy-up-log.txt submission/
| ls
```

Expected:

```
mv takes several sources and one destination, and the destination goes last. The archive folder now holds history.txt and the folders, and the three files are in submission.
```

7. Write your full name into `submission/name.txt`. Use one command.

```
echo "Aisha Khan" > submission/name.txt
```

Expected:

```
echo with > writes the line straight into a new file. You do not need touch first: > makes the file if it is not there.
```

8. List `submission` with the long details. Check that four files are there and that none of them is empty.

```
ls -l submission
```

Expected:

```
gallery-list.txt, aisha-count.txt, tidy-up-log.txt and name.txt. Every size column is bigger than 0. A 0 means you wrote the file with > twice and emptied it.
```

Remember

`history` belongs to the terminal window that you are in. Close the window before you save the file and the list starts again from nothing. Save `history.txt` before you close anything, and if you did lose it, say so when you hand in rather than typing a fake list: a short honest history is worth more than an invented one.

Hand in

Part 7 built your hand-in. This is the last thing you do, and it is two commands. Go up one folder, to `terminal-assignment`, and pack the whole `cs17-archive` folder into a zip file with your own name in the filename. `zip` is the partner of `unzip`, and `-r` tells it to include every folder inside:

```
cd ~/Desktop/terminal-assignment
zip -r submission-your-name.zip cs17-archive
```

Upload that zip file. Hand in this sheet with it.

Check this list before you pack the zip. Every line is something you can check with `ls`.

- `gallery` holds the six camera photographs and the `competition-2026` folder, and the screenshot is gone.
- `gallery/competition-2026` holds the three competition entries.
- `backup` holds all four experiment logs, and `logs` still holds them too.
- `sorted/notes` holds `last-year-notes.txt`, the copy of the Term 2 report and `camera-settings-backup.txt`.
- Four folders that came in the zip are gone: `documents/drafts`, `documents/temp`, `inbox/old-phone-photos` and `inbox/Unsorted Camera Dump`.
- `history.txt` is in `cs17-archive` itself, not in `submission`, and it is not empty.
- `submission` holds four files. One of them has your name.
- Every question, the map you copied in Part 1 and every table has an answer.

Command reference

<code>pwd</code>	show the folder that you are in
<code>ls</code>	list the contents of this folder
<code>ls -l</code>	list with the sizes and the dates
<code>ls -a</code>	list the hidden entries too
<code>cd</code>	change folder
<code>mkdir</code>	make a folder
<code>mkdir -p</code>	make every folder in the path that is missing
<code>touch</code>	make an empty file
<code>cat</code>	show the contents of a file
<code>cp</code>	copy a file
<code>mv</code>	move a file, or rename it
<code>rm</code>	delete a file. This is permanent
<code>rmdir</code>	delete an empty folder
<code>rmdir -p</code>	delete a folder and its parents, while each one is empty
<code>echo</code>	show text on the screen
<code>clear</code>	clear the screen
<code>history</code>	show the commands that you ran in this window
<code>head -n</code>	show the first n lines of a file
<code>tail -n</code>	show the last n lines of a file
<code>cowsay</code>	draw a cow saying something. It reads a pipe too
<code>grep</code>	show the lines of a file that contain a word
<code>wc</code>	count the lines, words and bytes
<code>wc -l</code>	count the lines only
<code>unzip</code>	unpack a zip file
<code>zip -r</code>	pack a folder and everything inside it into a zip file
<code>man</code>	open the manual of a command. Press q to quit
<code>--help</code>	ask a command to list its options
<code>></code>	write output into a file and replace the contents
<code>>></code>	add output to the end of a file
<code> </code>	send the output of one command into another command
<code>*</code>	match any text here
<code>..</code>	the folder above this one
<code>~</code>	your home folder
<code>"name with spaces"</code>	one name, not several arguments
<code>Tab</code>	finish the name that I started to type
<code>Up / Down</code>	move through the commands that you ran